

Coding for Artificial Intelligence

Module 1, Week 3-4: Sequence Manipulation

Problem Set 3

List Manipulation, Slicing, and Multi-Assignment Unpacking

Lab Project: The Multi-Dimensional Packet Scrambler

Course	Coding for AI (Module 1, Week 3) Introductory computing laboratories
Author	Dr. Prateek, Assistant Professor, School of AI
Format	In-class Practicals, 1 hour 30 minutes total, inclusive of write-up and submission
Total Marks	100

Read this document in full before you begin writing code.

Contents

1	Objective	2
2	The Problem: The Multi-Dimensional Packet Scrambler	2
3	Technical Requirements	2
3.1	Stage 1: Input Validation and Test Data	2
3.2	Stage 2: The “Middle-Out” Swap	2
3.3	Stage 3: In-Place Correction	3
3.4	Stage 4: Memory Integrity Check	3
4	Example	3
5	Constraints (“Standard”)	4
6	Edge Cases to Handle	4
7	Suggested Time Allocation	4
8	Deliverables	4
	Stretch Goal (Optional, Ungraded)	4
	Academic Integrity	5

1 Objective

Earlier problem sets dealt with single values — one name, one temperature, one derived index. This practical shifts focus to **sequences**: you will manipulate an entire list as a structured unit, using slicing, in-place mutation, and unpacking, without relying on any external library.

By the end of this practical you should be able to:

- (i) use **extended slicing** (including the **step** parameter) to rearrange a sequence non-destructively,
- (ii) distinguish operations that create a **new list** from operations that mutate a list **in place**, and
- (iii) use **multiple assignment unpacking** to decompose a sequence into named parts.

Time budget. This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 7.

2 The Problem: The Multi-Dimensional Packet Scrambler

In high-performance computing, data sequences are frequently rearranged, tagged, and cleaned without the overhead of importing external libraries — the built-in list operations are already efficient enough for this class of task. You are required to build a **Data Rotation Engine** that takes a list representing a data packet and puts it through a fixed pipeline of transformations: a structural rotation, an in-place correction pass, and a final integrity check.

3 Technical Requirements

Your script, `packet_scrambler.py`, must implement the following four stages, **in order**, and must print the state of the list after each stage (its **Trace Log** — see Section 8).

3.1 Stage 1: Input Validation and Test Data

1. Define your data packet as a list literal, hard-coded at the top of your script, containing **at least 10 integer elements**, including **at least two occurrences of the value 0** (these are needed to exercise Stage 3).
2. Write a small validation check that uses **short-circuit logic** to confirm the packet is both non-empty and long enough before any processing begins:

```

1 if packet and len(packet) >= 10:
2     print("Validation passed. Processing packet...")
3 else:
4     print("Validation failed: packet is empty or too short.")

```

Because **and** short-circuits, `len(packet)` is never evaluated on an empty list in a way that could cause problems — though note that `len([])` is perfectly legal in Python and simply returns 0. The real purpose of the **and** here is readability and intent: it states plainly that both conditions must hold.

3.2 Stage 2: The “Middle-Out” Swap

1. Compute the midpoint with `midpoint = len(packet) // 2` and divide the packet into two halves using slicing: `front_half = packet[:midpoint]` and `back_half = packet[midpoint:]`.
2. Build a new list in which the back half becomes the new front half, **reversed using the slice step parameter** (`back_half[::-1]`), followed by the untouched front half:

```

1 scrambled = back_half[::-1] + front_half

```

3. **Verify non-destructiveness.** Slicing always returns a *new* list object; it never modifies the list being sliced. Confirm this for yourself with `id()`:

```
1 print(id(packet) == id(front_half))    # Expect: False
```

If your original `packet` were to change after this stage, it would indicate that you had accidentally mutated it rather than sliced it — this check is your safeguard against that class of bug.

3.3 Stage 3: In-Place Correction

Work on the `scrambled` list only. The original `packet` must remain untouched from this point onward.

1. **Sync-Bit insertion.** Compute `middle_index = len(scrambled) // 2`. If the value at that index is an integer, insert the string "SYNC-BIT" immediately after it using `.insert()`:

```
1 middle_index = len(scrambled) // 2
2 if type(scrambled[middle_index]) is int:
3     scrambled.insert(middle_index + 1, "SYNC-BIT")
```

Use `type(x) is int`, **not** `isinstance(x, int)`, **for this check**. In Python, `bool` is a subclass of `int`, so `isinstance(True, int)` evaluates to `True`. If your `packet` ever contained a boolean value at the middle index, `isinstance()` would misclassify it as an integer; `type(x) is int` does not have this problem.

2. **Zero removal.** Remove *every* occurrence of the value 0 from `scrambled` using a `while` loop combined with `.remove()`:

```
1 while 0 in scrambled:
2     scrambled.remove(0)
```

Do not attempt this with a `for` loop that iterates over `scrambled` while also calling `.remove()` inside it — mutating a list while iterating over it with a `for` loop silently skips elements, because the loop's internal index advances even as the list shrinks underneath it. The `while 0 in scrambled` pattern avoids this entirely, re-scanning the list fresh on every iteration.

3.4 Stage 4: Memory Integrity Check

1. Print the original `packet` and the final `scrambled` list side by side, so the transformation is visible at a glance.
2. **Constraint:** use multiple assignment unpacking to isolate the header and footer of the final packet:

```
1 first, *middle, last = scrambled
2 print(f"Header: {first} Footer: {last} Body length: {len(middle)}")
```

The `*middle` collects every element that is not the first or the last into a new list, regardless of how many elements remain after Stage 3 removed the zeros.

4 Example

The following packet lets you check your implementation stage by stage.

Stage	List State
Initial packet	[5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
After Stage 2 (scrambled)	[3, 0, 19, 7, 34, 5, 12, 0, 8, 21]
After Sync-Bit insertion	[3, 0, 19, 7, 34, 5, "SYNC-BIT", 12, 0, 8, 21]
After zero removal	[3, 19, 7, 34, 5, "SYNC-BIT", 12, 8, 21]

With this final list, unpacking gives `first = 3`, `last = 21`, and a `middle` of length 7. The original 10-element `packet` should still read exactly [5, 12, 0, 8, 21, 34, 7, 19, 0, 3] when you print it again at the end — if it does not, a mutation has leaked into it somewhere in Stages 2–3.

5 Constraints (“Standard”)

1. **PEP 8 Compliance:** `snake_case` for all variable names; one space around every operator.
2. **No Aliasing:** every intermediate list (`front_half`, `back_half`, `scrambled`) must be produced by slicing or concatenation, never by direct reference assignment (`back_half = packet` is *not* slicing and shares the same underlying list).
3. **Docstring Quality:** a professional header docstring describing the four-stage pipeline and your author information.
4. **Trace Log Clarity:** each printed stage should be clearly labelled (e.g., “--- Stage 2: Middle-Out Swap ---”) so the transformation sequence is readable without referring back to this document.

6 Edge Cases to Handle

1. **Odd-length packets.** If `len(packet)` is odd, `front_half` and `back_half` will not be the same length. Confirm your slicing still produces a `scrambled` list of the correct total length.
2. **No integer at the middle index.** If, after Stage 2, the value at `middle_index` happens not to be an integer (for instance, if you test with a packet containing a float or a string), your program must skip the `.insert()` step cleanly rather than raising an error.
3. **A packet with no zeros.** Your `while` loop must terminate immediately and harmlessly if 0 is never in `scrambled` to begin with.

7 Suggested Time Allocation

Phase	Description	Duration
Implementation	Stages 1–4, using the worked example to self-check	55 minutes
Testing	Edge cases from Section 6; confirm original packet is untouched	20 minutes
Documentation & Submission	Docstring, trace log capture, final write-up	15 minutes
Total		90 minutes

8 Deliverables

1. **Source Code:** A single file named `packet_scrambler.py`.
2. **Trace Log:** The console output of a full run, showing the list’s state after each of the four stages, clearly labelled — this may be pasted as text or submitted as a screenshot.
3. **Submission Documentation:** A short note (100–150 words), included either as an extended docstring or a separate paragraph, explaining how your Stage 4 memory-integrity check confirms that Stages 2–3 never mutated the original `packet`.

Stretch Goal (Optional, Ungraded)

If you finish early, try generalising your script into a function `scramble(packet)` that returns the final list rather than printing it, and write two or three of your own test packets — including one with an odd length and one with no zeros at all — to confirm your function handles every edge case in Section 6 without modification.

Academic Integrity

You may discuss approaches with peers during the supervised practical session, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute's academic integrity policy.